

Service availability and failover

Follow-along: keep a service replying while the machine serving it dies

Overview

Two machines in this lab hold the same website, and a third sits in front of them and decides which one replies to each request. That third machine is the only address the client knows. When one of the two behind it stops working, the client is supposed to notice nothing: same address, same page, same status code. Whether that actually happens is not a property of the design. It is a property of four settings on the proxy, and this lab is those four settings.

The interesting part is which setting does what. It is tempting to assume the health check is what keeps the service up, because a health check is what notices a dead backend. It is not: a check that runs once a second cannot help a request that was sent half a second after the backend died. What saves that request is the retry policy, and what makes the retry cheap or expensive is the connect timeout. You will break the same backend three ways, and a different one of those settings is what holds each time.

This lab is about one service behind one proxy on one machine. It does not cover the other half of availability, which is what happens when the proxy itself dies or the whole site is unreachable; that needs a second proxy, a shared address and DNS or routing to move traffic between sites, and none of those is here. Nor does it cover what the backends themselves hold: both serve a fixed page, so nothing has to be kept in step between them.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

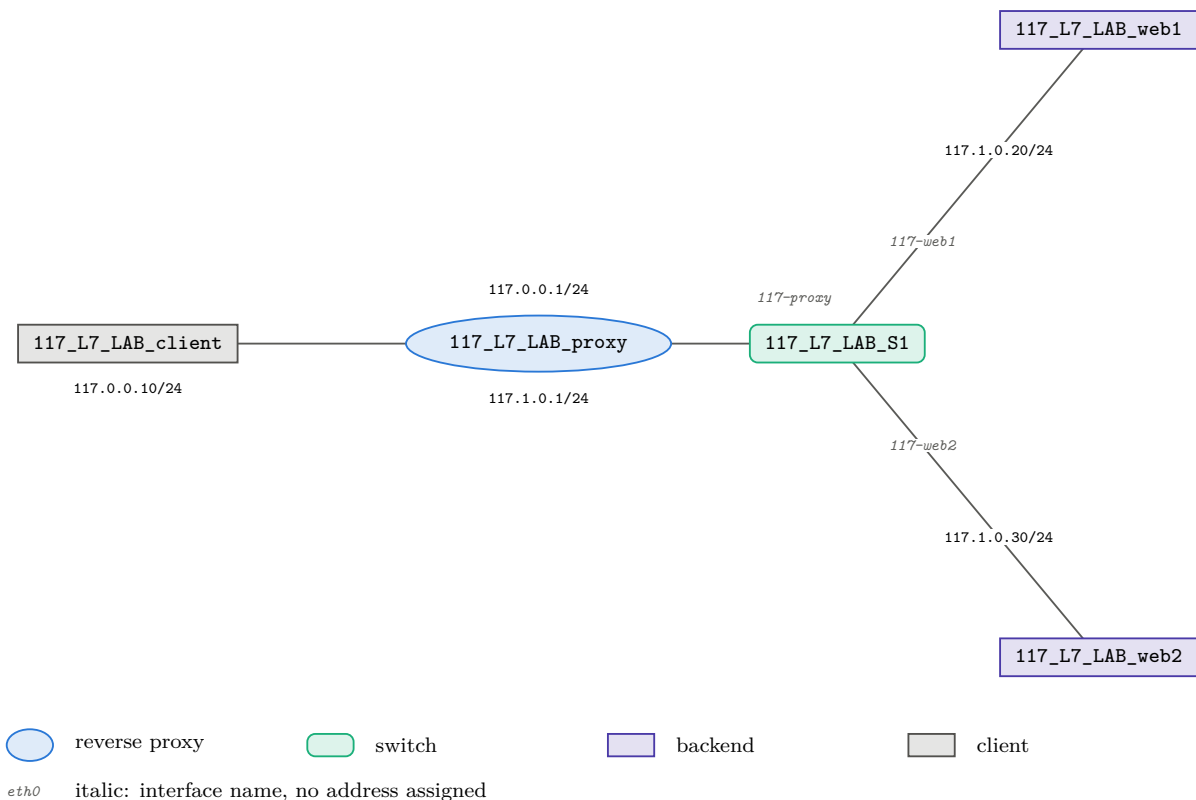
The root menu lists one category per row. Select **Host and service management**, then **Service availability and failover**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: what the proxy is configured to do, which backends it is currently sending requests to, and what the client got for its last twenty requests. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.

- **Shell command for a node** lists every node (client, proxy, web1, web2) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. You will want two terminals open for most of this lab: one on the client and one on whichever machine you are about to break.
- **Reset to baseline** throws away the configuration you wrote on the proxy, repairs both backends, and stops the proxy, putting the lab back where Part 1 starts it without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Five containers on two segments. The client sits alone on the front segment and is where every number in this lab is measured. The proxy has an interface on each segment. Both backends and the proxy's second interface share a switch on the back segment.

The proxy does not forward packets between the two segments: `net.ipv4.ip_forward` is 0 on it. The client therefore has no path to a backend at all. Every connection a backend ever receives is one the proxy opened itself, on the client's behalf, which is what makes the proxy the only place a backend's health can be observed from.

Container	Role	Address	Listening on
117_L7_LAB_client	the client	117.0.0.10	nothing
117_L7_LAB_proxy	the proxy, front side	117.0.0.1	tcp/80 once you configure it
	the proxy, back side	117.1.0.1	
117_L7_LAB_web1	backend	117.1.0.20	tcp/80
117_L7_LAB_web2	backend	117.1.0.30	tcp/80
117_L7_LAB_S1	switch, one bridge	none	none

Each backend runs `lighttpd` and serves two things. Its index page is one word, its own name, so a reply tells you which machine produced it. Separately, `/health` returns 200 with a body of OK. The two are different files, and Part 2 turns on the difference.

The proxy runs `haproxy` 2.8. Its configuration is at `/etc/haproxy/haproxy.cfg` and arrives holding a `global` section, a `defaults` section, and nothing else. The tools you need on the client are `curl` and `nmap`; `vi` and `nano` are on every machine.

Assessment

This handout carries **eighteen** graded questions, numbered in order and placed at the step that produces the evidence for them. Answer them in a separate document. Several ask you to record a number you measured; record the number you got, not the one you expected, and if the two differ say so.

Part 1: the pool

Definition: Reverse proxy

A server that accepts a client's connection, opens a separate connection of its own to one of several machines that hold the content, and copies the reply back. The client connects to it and never learns which machine produced the reply. It is called reverse because a forward proxy is chosen by the client to reach the internet, while this one is chosen by the operator to stand in front of a service. In this lab the proxy is 117.0.0.1 and the client only ever asks it.

Definition: Backend and pool

A **backend** is one of the machines holding the content. The **pool** is the set of them the proxy is willing to send a request to. HAProxy uses the word **backend** for the configuration section that names the whole set, and **server** for one line inside it, so this lab's pool is a **backend** section called `be_web` holding two **server** lines.

Definition: Load balancing algorithm

The rule the proxy uses to pick which server in the pool gets the next request. **roundrobin** takes each server in turn, so with two healthy servers and twenty requests each server gets ten. Other rules pick by number of open connections (**leastconn**) or by a hash of the client's address (**source**), which sends one client to the same server every time.

Open a shell on the proxy and look at what is there:

```
# 117_L7_LAB_proxy: the machine the whole lab is written on
cat /etc/haproxy/haproxy.cfg
```

The **global** section holds one thing that matters here, a **runtime API** socket, which Part 5 uses and which the Status action reads to report the pool. The **defaults** section holds three timeouts. There is no **frontend** and no **backend**, so there is nothing telling HAProxy to listen anywhere or where to send anything.

Definition: Runtime API

A UNIX socket HAProxy listens on for one-line commands, so a running proxy can be queried and changed without editing its configuration file or restarting it. **show stat** returns the state of every server as a CSV document; **set server** changes one server's state. The socket in this lab is **/run/haproxy/admin.sock**, declared **level admin** so that it accepts commands that change things rather than only report them.

That file is valid: **haproxy -c** accepts it, and **haproxy -D** would start a daemon from it. The daemon would bind nothing, because a listening socket comes from a **frontend** and there is no **frontend**. Add both sections. Open the file and append:

```
# 117_L7_LAB_proxy: append to /etc/haproxy/haproxy.cfg
frontend fe_web
    bind 117.0.0.1:80
    default_backend be_web

backend be_web
    balance roundrobin
    server web1 117.1.0.20:80
    server web2 117.1.0.30:80
```

Check the file before loading it, then start the daemon. Getting into the habit of the first command is the point of it: a reload with an invalid file leaves whatever was already running in place, and a start with one leaves nothing running at all.

```
# 117_L7_LAB_proxy
haproxy -c -f /etc/haproxy/haproxy.cfg
haproxy -D -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid
```

Now measure it from the client. Each backend serves its own name, so twenty requests tally into two counts:

```
# 117_L7_LAB_client: one command, run from your own terminal
for i in $(seq 1 20); do curl -s http://117.0.0.1/; done | sort | uniq -c
```

Question 1. Before you run that loop, say how **balance roundrobin** will divide twenty requests between two healthy servers. Then run it and give the two counts you actually got.

The client has no route to a backend, and it is worth seeing what that looks like rather than taking it on trust. Scan both a backend and the proxy from the client:

```
# 117_L7_LAB_client
nmap -Pn -n -p 80 117.1.0.20    # web1, on the back segment
nmap -Pn -n -p 80 117.0.0.1    # the proxy, on the front segment
```

Definition: closed, filtered, and open

The three states `nmap` reports for a TCP port, and they are conclusions about replies rather than facts about the machine. **open** means a listener replied to the probe with a SYN-ACK. **closed** means the host replied with a TCP reset, which says the packet reached a machine and no process was listening on that port. **filtered** means nothing replied at all, so the probe was discarded somewhere and the sender cannot tell where or by what. The difference between the last two carries the whole of Parts 3 and 4.

Question 2. Give the state `nmap` reported for each of the two scans above, and name the setting on the proxy that produces the difference between them. Then trace the probe aimed at `web1`: say which machine received it, what that machine did with it, and why the state `nmap` reported is the only one it could report from what came back.

Run the Status action and look at the pool table. Both servers are listed, and the status column for each says `no check` rather than `UP`.

Question 3. HAProxy reports a server it has never probed as `no check`, and still sends it requests. State what HAProxy is assuming about such a server, and what that assumption costs the client the moment the server stops working.

Part 2: the health check

The pool works and the proxy is guessing. It has no way to find out whether a server is able to serve, so it keeps handing every second request to whatever is on the second `server` line whether or not anything is there. A health check is the proxy asking, on a schedule, before a client has to find out.

Definition: Active and passive health checks

An **active** check is a request the proxy sends on its own schedule, to an address and path it was configured with, whose reply decides whether the server stays in the pool. A **passive** check draws the same conclusion from the client traffic already flowing, by counting how many real requests failed. Active checking finds a broken server with no client involved and costs a request per server per interval; passive checking costs nothing extra and cannot notice anything until a client has already been served badly. HAProxy does both; this lab configures the active one.

Definition: inter, fall and rise

The three numbers that shape a check. **inter** is how long HAProxy waits between checks of one server. **fall** is how many checks in a row must fail before a server that was in the pool is taken out. **rise** is how many must succeed in a row before a server that was out is put back. With `inter 1s fall 2`, a server that starts failing is out somewhere between one and two seconds later, depending on where in the interval the failure began.

Change the `backend` section so each server is checked, and tell HAProxy what a check is:

```
# 117_L7_LAB_proxy: /etc/haproxy/haproxy.cfg, replacing the backend section
backend be_web
    balance roundrobin
    option httpchk
    http-check send meth GET uri /health
    http-check expect status 200
    server web1 117.1.0.20:80 check inter 1s fall 2 rise 3
    server web2 117.1.0.30:80 check inter 1s fall 2 rise 3
```

The **check** keyword on the server line is what makes HAProxy probe that server at all. The three **http-check** lines only describe what a probe is; without **check** on the server line they are never sent to anything.

Reload rather than restart. The second command hands the listening socket to a new process and tells the old one to finish the requests it has already accepted:

```
# 117_L7_LAB_proxy
haproxy -c -f /etc/haproxy/haproxy.cfg
haproxy -D -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -sf $(cat \
/run/haproxy.pid)
```

Read the pool through the runtime API. This is the same document the Status action parses:

```
# 117_L7_LAB_proxy
echo "show stat" | socat /run/haproxy/admin.sock stdio | cut -d, -f1,2,18,37
```

```
# 117_L7_LAB_proxy prints this; it is output, not commands to type
be_web,web1,UP,L7OK
be_web,web2,UP,L7OK
```

Definition: Check result codes

What HAProxy reports for the last completed check, and the four this lab produces. L7OK is a check that got the status code it was told to expect. L7STS is a check that got a reply with a *different* status code, so the service is running and replying and the check still failed. L4CON is a connection the far end refused. L4TOUT is a connection attempt that got no reply at all before the timeout. The two L4 results are about whether a TCP connection could be established; the two L7 results only happen once one was.

Now break the health endpoint on web1 without touching its website. The two are separate files:

```
# 117_L7_LAB_web1: shell in on a second terminal
mv /var/www/localhost/htdocs/health /var/www/localhost/htdocs/health.off
```

Wait a few seconds, then read the pool again, and check what web1 is actually doing. The proxy is the only machine with a route to it:

```
# 117_L7_LAB_proxy
echo "show stat" | socat /run/haproxy/admin.sock stdio | cut -d, -f1,2,18,37
curl -s http://117.1.0.20/
curl -s -o /dev/null -w '%{http_code}\n' http://117.1.0.20/health
```

Question 4. Give the status and check result web1 now reports, and the two things the `curl` commands returned. State what the check established about web1 and what it did not.

Question 5. Compute the longest a server can keep receiving requests after its health endpoint starts failing, from the `inter` and `fall` values you configured. At five requests per second split evenly over two servers, how many requests reach that server inside that window, and what status code does the client get for them?

Question 6. `fall` is 2 and `rise` is 3, so it takes fewer failures to remove a server than successes to bring it back. Say what the cost is of getting each of those two decisions wrong: taking out a server that was working, and returning one that is not.

Run the request loop from the client again while web1 is out, then put the health endpoint back:

```
# 117_L7_LAB_client
for i in $(seq 1 20); do curl -s http://117.0.0.1/; done | sort | uniq -c
```

```
# 117_L7_LAB_web1
mv /var/www/localhost/htdocs/health.off /var/www/localhost/htdocs/health
```

Part 3: the request that was already in flight

The health check in Part 2 removed a server that had stopped passing checks, and no client saw a failure, because the server was still serving. That is the easy case. This part is the case a health check cannot cover on its own.

A check running every second tells you about the server as it was at most a second ago. A request sent in the meantime is sent to a server the proxy still believes is fine. If the process behind it has gone, the connection fails, and by then the request exists and something has to reply to it. Whether the client sees that failure is decided by what HAProxy does with a connection that could not be established.

Definition: Retry and redispach

A **retry** is HAProxy making a second attempt to reach a server after an attempt failed, inside the same client request; the client is still waiting and knows nothing about it. **retries** sets how many extra attempts are allowed. On its own, every retry goes back to the same server. **Redispach** is the separate instruction to send a retry to a *different* server in the pool, which is what turns a retry from a second try at a machine that just failed into a first try at one that has not.

The measurement needs traffic running while the backend dies, so the loop moves into the background and writes each result to a file. `-w` makes `curl` print the status code and the time the request took; `-o /dev/null` throws the page away, because this loop is counting failures rather than reading content.

```
# 117_L7_LAB_client: starts a 40-request loop in the background and returns your \
prompt
rm -f /tmp/run.log
( for i in $(seq 1 40); do
    curl -s -m 5 -o /dev/null -w "%{http_code} %{time_total}\n" http://117.0.0.1/ \
    >> /tmp/run.log
    sleep 0.2
done ) &
```

Forty requests two-tenths of a second apart is eight seconds of traffic, which is enough time to switch terminals and kill something. First, turn retries off explicitly, so the only thing standing between a dead backend and the client is the health check you already configured. Add one line to `defaults` and reload:

```
# 117_L7_LAB_proxy: add to the defaults section
retries 0
```

```
# 117_L7_LAB_proxy
haproxy -c -f /etc/haproxy/haproxy.cfg
haproxy -D -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -sf $(cat \
/run/haproxy.pid)
```

Start the loop on the client, wait about three seconds, and kill the web server on `web1` from a second terminal. `lighttpd` writes its process id to a file, which is a more reliable handle than its name:

```
# 117_L7_LAB_web1
kill -9 $(cat /run/lighttpd.pid)
```

When the loop finishes, tally it:

```
# 117_L7_LAB_client
awk '{print $1}' /tmp/run.log | sort | uniq -c
sort -k2 -g /tmp/run.log | tail -1      # the slowest request
```

Question 7. Give the number of requests that did not get a `2xx` status, and the status code they got instead. The health check was configured and working throughout. Explain why it did not prevent them.

Question 8. Work out from `inter`, `fall` and the loop's request rate how many requests should have been sent to `web1` between the moment it died and the moment the check removed it. Compare that with the number you measured and account for any difference.

Bring `web1` back and wait for it to return to the pool. `rise 3` means three consecutive good checks, so give it four or five seconds:

```
# 117_L7_LAB_web1
lighttpd -f /etc/lighttpd/lighttpd.conf
```

Now allow retries, but do not say where they should go. Replace the `retries 0` line with `retries 3`, reload, and run the same measurement again: start the loop, wait three seconds, kill `lighttpd` on `web1`.

Before reading the result, one fact about how HAProxy paces a retry, because nothing you have configured predicts it. The 2.8 configuration manual states, under `retries`: “In order to avoid immediate reconnections to a server which is restarting, a turn-around timer of `min("timeout connect", one second)` is applied before a retry occurs on the same server.” The `timeout connect` in your `defaults` section is currently 5s, so that minimum is one second.

Question 9. Give the number of failed requests and the time the slowest request took. Using the turn-around rule above and the `retries` value you set, account for that time. Say what each of the retries was attempting and why none of them succeeded.

Retries alone bought nothing here and cost seconds, because a retry with no `redispatch` goes back to the machine that just refused the connection. Add the instruction that sends it elsewhere:

```
# 117_L7_LAB_proxy: the defaults section now reads
retries 3
option redispatch
```

Reload, revive `web1`, wait for it to rise, and run the same measurement a third time.

The manual’s `retry-on` entry states that the default condition for a retry is `conn-failure`, described as “retry when the connection or the SSL handshake failed”. So by default HAProxy retries only when it could not establish a connection to the server.

Question 10. Give the failed-request count and the slowest request for this run. Then explain, from the `retry-on` default above, why re-sending this particular request to a second server cannot cause the client’s request to be carried out twice, and name a kind of failure where a retry would not be safe in that way.

Question 11. The turn-around timer in the manual is applied “before a retry occurs on the same server”. Use that wording to explain why the slowest request in this run is so much shorter than the slowest request in the previous one, even though `retries` is 3 in both.

Part 4: when nothing replies at all

A killed process is the polite failure. Nothing is listening on the port, so the machine’s own network stack replies to the connection attempt with a TCP reset, immediately and without being asked to. HAProxy knows the attempt failed within a round trip and can act on it.

The other failure is silence: the packets are discarded and no reply of any kind comes back. That is what a crashed machine looks like, and a full connection table, and a firewall rule in the path. From the proxy’s side there is nothing to react to, only an absence, and the only thing that ends the attempt is a timer.

Definition: `timeout connect`

How long HAProxy will wait for a TCP connection to a server to be established before giving up on that attempt. It is separate from `timeout server`, which bounds how long an already established connection may stay idle waiting for the server to send something. `timeout connect` is the only one of the two that a server which never replies at all can reach.

Your configuration still has `retries 3` and `option redispatch`, which was enough for the killed process. Leave it exactly as it is and change the failure instead. On `web1`, drop every packet arriving for port 80 rather than stopping the service:

```
# 117_L7_LAB_web1
nft add table inet blackhole
nft 'add chain inet blackhole input { type filter hook input priority filter ; \
    policy accept ; }'
nft add rule inet blackhole input tcp dport 80 drop
```

Question 12. Before you run anything: your retry policy kept the failure count at zero when `web1`'s process was killed. Say whether you expect it to do the same now that `web1`'s packets are dropped instead, and give your reason.

Start the loop on the client, wait three seconds, then add the drop rule above. Tally the result the same way.

Question 13. Give the failed-request count and the slowest request. State the value of `timeout connect` in your `defaults` section and the value of `-m` on the client's `curl`. Write the inequality that has to hold between `timeout connect`, the number of attempts your `retries` setting allows, and the client's limit, for no request to fail.

Two changes make it hold. Shorten the connect timeout so a single attempt that gets no reply is cheap, and move the `redispatch` off the last retry. The manual's `option redispatch` entry describes the optional argument: "the default of `-1` preserves the historical behavior of `redispatching` on the last retry, a positive value of `1` would indicate a `redispatch` on every retry".

```
# 117_L7_LAB_proxy: the defaults section now reads
    timeout connect 200ms
    retries 3
    option redispatch 1
```

Reload, remove the drop rule, wait for `web1` to rise, then run the measurement once more with the drop rule added mid-loop:

```
# 117_L7_LAB_web1
nft delete table inet blackhole
```

Question 14. Give the failed-request count and the slowest request for the tuned run. The slowest request is close to one specific value from your configuration. Say which value, and which of the two changes you made is responsible for the request costing that once rather than three times.

Question 15. Read the check result for `web1` through the runtime API while it is blackholed, and compare it with the one you saw in Part 3 when its process had been killed. Give both codes, and say what the proxy received in each case that let it tell the two apart.

Remove the drop rule and let `web1` come back before going on.

Part 5: taking a server out on purpose

Everything so far has been a failure nobody chose. The commoner reason a backend leaves a pool is that somebody is about to work on it, and then the goal is different: not to react quickly, but to stop sending it work while it finishes what it already has.

Waiting for the health check to notice is the wrong tool for that. It is slow by design, it needs the server to actually start failing, and requests are lost in the window. The runtime API removes a server on command.

Definition: drain and maint

Two administrative states a server can be put into through the runtime API, both of which persist until an explicit `set server ... state ready` clears them. The 2.8 management guide describes **drain** as removing “the server from load balancing but still allows it to be checked and to accept new persistent connections”, and **maint** as disabling “any traffic to the server as well as any health checks”. So a drained server keeps being probed and can still be reached by a client that a persistence rule pinned to it; a server in maintenance is reached by nothing and probed by nothing. This lab configures no persistence, so a drained server here receives nothing at all.

Read `web1`’s session count, run twenty requests, and read it again, so you know what the number does while nothing is wrong. The `stot` column is the total sessions a server has handled since HAProxy started:

```
# 117_L7_LAB_proxy
echo "show stat" | socat /run/haproxy/admin.sock stdio | cut -d, -f1,2,18,8
```

```
# 117_L7_LAB_client
for i in $(seq 1 20); do curl -s -o /dev/null http://117.0.0.1/; done
```

Now drain `web1` and repeat both commands:

```
# 117_L7_LAB_proxy
echo "set server be_web/web1 state drain" | socat /run/haproxy/admin.sock stdio
```

Question 16. Give `web1`’s session count before and after the twenty requests you sent while it was draining, and its status. Then state what a drain does for a request that is already in flight to that server which the health check removing it after `fail` failures does not.

Question 17. You need `web1` out of service for twenty minutes of work, during which its web server will be stopped and started several times. Give the runtime API command you would use, and give the two things in the definition above that **drain** would still be doing to `web1` for those twenty minutes.

Put `web1` back, and confirm it returns:

```
# 117_L7_LAB_proxy
echo "set server be_web/web1 state ready" | socat /run/haproxy/admin.sock stdio
```

The last planned interruption is the proxy itself. Every reload you have run in this lab used

`-sf`, and it is worth measuring what that flag is doing, because a reload touches every request rather than one server's share. Start the loop on the client, and reload the proxy three seconds in:

```
# 117_L7_LAB_proxy
haproxy -D -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -sf $(cat \
/run/haproxy.pid)
```

Question 18. Give the failed-request count across the reload. The old process was still holding accepted requests when the new one started. State what `-sf` told it to do with them, and what the client would have seen had the old process been killed outright instead.

Checks: what “done” means

Run the Status action. It reports the five parts against the configuration currently on the proxy and the last twenty requests from the client:

- **Part 1** the pool balances: both backends served a share of the last twenty requests.
- **Part 2** both servers carry a check, so neither is reported as `no check`.
- **Part 3** `retries` is at least 1 and option `redispatch` is present, so a failed connection is re-sent to a different server.
- **Part 4** every attempt the retry policy allows fits inside the client's own limit, which Status states as arithmetic on your `timeout connect` and `retries` values.
- **Part 5** a server is currently held out of the pool by `drain` or `maint`.

Status cannot report the number this lab is actually about. Measuring the failure count across a backend dying means causing the failure, so that measurement stays with you: the request loop on the client, and something broken on a backend while it runs.

Safety note

Every container in this lab sits on an isolated Open vSwitch bridge and a point-to-point link with no route off the host. The proxy holds the only pair of interfaces that touch both segments and does not forward between them. The `nftables` rule in Part 4 is written inside a backend container and affects only traffic arriving at that container. Nothing here can reach the machine you are running it on, or anything that machine can reach.

Tearing the lab down

Remove the lab when you are finished with it. Use the **Teardown** action in the TUI, or run `scripts/teardown.sh` directly. It removes this lab's five containers and nothing else.